Applied Soft Computing Journal 187 (2026) 114318

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

A hybrid evolutionary framework assisted by reinforcement learning for mixed multi-objective optimization features

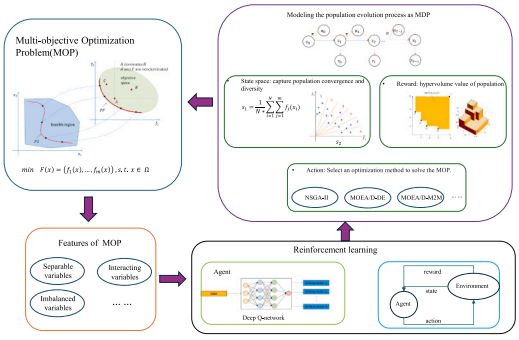
Yanhui Tang [a], Hui Li [a,*], Haotian Zhang [b], Jianyong Sun [a]

[a] School of Mathematics and Statistics, Xi'an Jiaotong University, Xi'an, 710049, Shaanxi, China
[b] Frontier Institute of Science and Technology, Xi'an Jiaotong University, Xi'an, 710049, Shaanxi, China

HIGHLIGHTS

- Integrates reinforcement learning with multi-objective evolutionary algorithm to dynamically select the best optimizer during search stages.
- A novel set of multi-objective optimization problems with mixed features is proposed.
- Our method finds high-quality solutions faster than existing methods on benchmarks, especially for multi-objective optimization problems with mixed features.

GRAPHICAL ABSTRACT



ARTICLE INFO

Keywords:
Multi-objective optimization
Evolutionary algorithms
Reinforcement learning
Deep Q-Network

ABSTRACT

Many real-world problems involve multi-objective optimization. Although traditional multi-objective evolutionary algorithms (MOEAs) have achieved considerable success in solving certain types of multi-objective problems (MOPs), they often encounter difficulties when dealing with problems exhibiting multiple complex features, and struggle to find high-quality non-dominated solutions. To address this issue, this paper proposes a reinforcement learning-based hybrid evolutionary multi-objective optimization framework (RL-MOEA), designed to handle MOPs with mixed features. RL-MOEA integrates reinforcement learning with MOEAs to improve adaptability during optimization. This framework models the evolutionary process as a Markov Decision Process (MDP), employing two state representations to capture population distribution and using hypervolume (HV) as a reward signal. Through reinforcement learning, intelligent agents are trained to dynamically select the most suitable optimal method at different evolutionary stages. Computational experiments indicate that RL-MOEA outperforms several state-of-the-art MOEAs on various benchmark problems, achieving optimal inverted generational distance (IGD) values on MOPs with mixed features. The proposed method also exhibits better convergence and diversity under comparable computational budgets. These results demonstrate the capability of RL-MOEA to efficiently solve a wide range of complex MOPs.

1. Introduction

Multi-objective optimization problems (MOPs) frequently arise in real-world applications, such as multi-objective vehicle routing problems and portfolio optimization [1–4]. Formally, MOPs require the simultaneous optimization of multiple conflicting objectives [5]. Due to the partial ordering induced by Pareto dominance, no single solution

* Corresponding author.
Email address: lihui10@xjtu.edu.cn (H. Li).

https://doi.org/10.1016/j.asoc.2025.114318
Received 24 June 2025; Received in revised form 27 September 2025; Accepted 22 November 2025
Available online 22 November 2025
1568-4946/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

can optimize all objectives. The primary goal of solving MOPs is to search for a set of Pareto-optimal solutions that mathematically represent the trade-offs between objectives. This allows decision-makers to choose their preferred solution according to their specific requirements.

Although gradient-based methods demonstrate efficiency in single-objective optimization, they exhibit significant limitations when applied to MOPs. This is mainly due to their inability to generate multiple Pareto-optimal solutions in a single run. For instance, weighted sum methods require predefined preference weights and yield only one solution per run, limiting their utility in scenarios requiring comprehensive trade-off analysis [6]. Similarly, scalarization techniques like the ϵ-constraint method struggle with high-dimensional objective spaces, where manual parameter tuning becomes intractable [7]. These limitations have driven the development of population-based approaches, particularly multi-objective evolutionary algorithms (MOEAs), which leverage stochastic search to approximate Pareto fronts through iterative population evolution.

Over the past three decades, MOEAs have become the primary approach for solving MOPs. The design of MOEAs considers various features of MOPs, such as the complex geometric shapes of Pareto fronts [8–10], interactions between variables [11,12], large number of decision variables [13,14], multi-modality [15,16], and so on. In the research area of MOEAs, there exist three major algorithmic frameworks: Pareto-based, decomposition-based, and indicator-based approaches. The Pareto-based framework, exemplified by NSGA-II [17], employs nondominated sorting to rank solutions and crowding distance to preserve diversity. In contrast, MOEA/D [18] adopts a decomposition strategy, decomposing a multi-objective problem into multiple single-objective subproblems. The indicator-based approach is represented by IBEA [19], which selects individuals based on their contribution to the hypervolume indicator of the population. Each framework offers distinct advantages depending on specific problem features, but each also has inherent limitations in universally addressing all types of MOP features.

In recent years, continuous innovation has led to the development of various efficient algorithms, many drawing inspiration from unique natural or physical phenomena. For example, the Multi-objective Thermal Exchange Optimization (MOTEO) algorithm [20] is based on thermodynamic principles. It updates candidate solutions by simulating the process of objects reaching thermal equilibrium through heat exchange, effectively enhancing convergence. Inspired by the foraging behavior of slime molds, the Multi-objective Slime Mold Algorithm (MOSMA) [21] utilizes their oscillation patterns and adaptive network to adjust weights. This effectively balances global exploration with local exploitation while maintaining diversity. Another nature-inspired method, the Multi-objective Ant Lion Optimizer (MOALO) [22], simulates the hunting mechanism of antlions. It simulates the process of antlions digging traps to prey on ants, resulting in robust global search performance.

Moreover, MOEAs have been extensively applied to real-world problems. For example, the recently introduced Multi-objective Weighted Average Algorithm (MOWAA) [23] employs an adaptive weighted average strategy to dynamically guide the population's search direction. This approach has been shown to outperform well-established algorithms such as NSGA-II and MOEA/D in challenging engineering applications like truss design. Similarly, the Grey Wolf Optimizer (GWO) has been effectively used for selection of cloud services [24], leveraging its strong global search capability and rapid convergence. The Group Search Optimizer (GSO) [25] proves effective for blockchain-based IIoT systems, efficiently identifying Pareto-optimal solutions for multi-modal MOPs.

The MOEA research community has developed several test suites to systematically evaluate algorithm performance, including widely used sets such as ZDT [8], DTLZ [9], WFG [26], and UF [12]. These test suites exhibit notable differences in their features. For example, ZDT problems are designed with separable decision variables, while UF problems involve interacting variables. However, real-world optimization problems often integrate multiple complex features that may only partially align with existing benchmarks or introduce new characteristics not yet covered [27,28]. As emphasized by the "No Free Lunch Theorem" [29], no single algorithm can efficiently solve all types of problems. Therefore, developing hybrid algorithms capable of adapting to diverse and complex problem features remains a key research direction in multi-objective optimization.

Many researchers have developed hybrid algorithms to overcome the limitations of a single search strategy in balancing diversity and convergence. Recent innovations include adaptive differential evolution (DE) strategies that apply distinct operators to different gene positions of individuals, thereby enhancing exploration across the solution space [30]. Another approach integrates Particle Swarm Optimization (PSO) with DE, leveraging PSO to accelerate convergence while using DE to maintain diversity [31]. These hybrid frameworks have also shown significant promise in combinatorial optimization. For example, a hybrid NSGA-II enhanced with local search has been successfully applied to vehicle routing problems, achieving an improved balance between global exploration and local exploitation [32]. Similarly, a three-phase hybrid approach has been proposed for bi-objective traveling salesman problems, which employs different algorithms at various stages to obtain a better trade-off between convergence and diversity [33].

Despite considerable advancements, hybrid algorithms still face two fundamental challenges. First, existing hybrid algorithms are often designed for specific problem features and have limited capability in handling MOPs that combine multiple complex features. This narrowness often leads to suboptimal performance when applied to real-world problems, which often incorporate diverse features. Second, many heuristic-driven designs lack theoretical guarantees, resulting in performance instability when confronted with different Pareto front geometries or variable types.

Reinforcement learning (RL) provides an effective mechanism for adaptive operator and strategy selection in evolutionary algorithms through its state-action-reward framework. This capability allows RL to be effectively integrated with various optimization techniques to enhance their adaptability in complex environments. In single-objective optimization, RL has demonstrated its ability to automate parameter control for differential evolution [34] and hybridize search strategies in sequential optimization [35].

In the context of MOPs, RL-driven frameworks have shown promising results. For instance, Q-learning has been employed to develop dynamic MOEAs that adaptively select among knee-based, indicator-based, and center-based strategies [36]. Deep RL has further been applied to identify sparse variables in large-scale sparse MOPs [37], and to decompose MOPs into scalar sub-problems modeled via neural networks, enabling direct generation of Pareto-optimal solutions [38]. Additionally, Heidari et al. [39] utilized deep reinforcement learning to dynamically optimize task offloading strategies like target node selection and data volume allocation, thereby achieving goals such as low latency and energy efficiency. Another notable contribution is a Q-learning-based MOEA framework that adaptively switches between NSGA-II and MOEA/D-DE, demonstrating superior performance on problems with mixed variable types [40]. These advancements highlight the potential of RL in developing self-adaptive MOEAs capable of addressing various problem features.

Furthermore, there have been specific developments in research that combine RL with hyper-heuristic algorithms. Initially proposed by Cowling et al. [41], hyper-heuristics focus on developing heuristic selection mechanisms during the search process. RL has been successfully applied in hyper-heuristics, as demonstrated in studies [42], where it was used to solve exam timetabling problems, outperforming random selection heuristics. Recent developments include the use of deep RL proxies to select low-level heuristics in Capacitated Vehicle Routing Problems (CVRP) [43], as well as deep RL to enhance decision-making in adaptive search frameworks [44]. However, hyper-heuristic algorithms

are primarily tailored for specific combinatorial optimization problems and often lack generalizability to other domains, particularly MOPs with mixed features.

The integration of RL into optimization algorithms has shown significant promise for automating parameter control, adapting search strategies, and improving computational efficiency in both single and multi-objective problems. These advancements demonstrate the increasingly important role of RL in developing adaptive optimization frameworks capable of handling diverse and complex problem features.

Although considerable advances have been made in optimization algorithms, no single method has proven universally effective across all types of problems. Real-world multi-objective optimization problems often involve complex variable features [45]. Two particularly important and common features in such problems are variable interactions, which refer to dependencies and couplings among variables, and imbalanced variables, which exert asymmetric, directional, or nonlinear influences on the objectives. For example, in aircraft wing design [46], parameters such as sweep, camber, and thickness are highly interdependent. Under supersonic conditions, an optimal sweep range exists; deviation below this range leads to a sharp increase in drag. When addressing such complex engineering challenges, conventional optimization methods, which often rely on a single algorithm, struggle to perform effectively. This highlights the need for more adaptive and flexible optimization approaches.

To address these limitations, we propose RL-MOEA, a hybrid evolutionary framework based on reinforcement learning to solve MOPs with mixed features, which can adaptively select the most suitable algorithm according to the features of the problem. RL-MOEA models population evolution as a Markov Decision Process (MDP), where states capture convergence progress and population diversity, actions correspond to algorithm selection, and rewards reflect improvements in hypervolume. A Deep Q-Network (DQN) is employed to handle continuous state spaces, allowing for more detailed policy adaptation.

The implementation of RL-MOEA consists of two sequential phases. In the training phase, the DQN learns an optimal algorithm selection policy using a predefined benchmark problem. During the optimization phase, the trained agent dynamically selects algorithms in real time to solve MOPs. For experimental validation, three algorithms are integrated into the framework: NSGA-II, MOEA/D-DE [11], and MOEA/D-M2M [47]. These algorithms were selected due to their complementary strengths: NSGA-II performs well on problems with separable variables, MOEA/D-DE is capable of handling interacting variables, and MOEA/D-M2M excels in managing imbalanced problems. It is important to note that the framework is scalable and can easily incorporate additional algorithms. To comprehensively evaluate performance, this paper also constructs a set of test problems that incorporate these three feature types.

This study has made three key contributions:

- The proposed RL-MOEA framework intelligently combines multiple MOEAs via reinforcement learning, providing adaptive algorithm selection according to problem features.
- A novel test suite with mixed problem features is proposed to evaluate algorithm performance.
- Experimental results on test problems show that RL-MOEA achieves better performance than existing methods when solving MOPs with mixed problem features.

The remainder of this paper is organized as follows. Section 2 presents fundamental background knowledge, including MOPs, reinforcement learning, research motivation, and the proposed testing problems. Section 3 details the RL-MOEA architecture, including state-action definitions, reward mechanisms, and DQN implementation. The computational results are presented and analyzed in Section 4. Finally, Section 5 concludes the paper with key findings and potential future research directions.

2. Background

2.1. Basic concepts of multi-objective optimization

Multi-objective optimization problems (MOPs) typically involve multiple conflicting objective functions, and their mathematical form is usually as follows:

$$\begin{aligned} \text{minimize} \quad & F(x) = (f_1(x), \ldots, f_m(x)) \\ \text{s.t.} \quad & x \in \Omega \subseteq R^n \end{aligned} \tag{1}$$

where $f_i, i = 1, \ldots, m$ is the i-th objective function and Ω is the feasible region in the decision space. Due to the conflicting objectives, MOP does not seek a single optimal solution but rather explores a set of solutions, namely the Pareto optimal solution set, where each solution provides different trade-offs among various objectives.

For the two solutions x and y in $\Omega \subseteq R^n$, if and only if $f_i(x) \leq f_i(y), i = 1, \ldots, m$ and at least one j exists, such that $f_j(x) < f_j(y)$, it is said that x dominates y. If there is no solution in $\Omega \subseteq R^n$ that can dominate the solution x^*, then x^* is called a Pareto optimal solution. All Pareto-optimal solutions form a Pareto optimal set (PS), and the mapping of the PS in the objective space is called a Pareto front (PF).

Understanding these fundamental concepts provides a basis for analyzing the features of MOPs. In this paper, the term mixed feature refers to MOPs that incorporate multiple types of features within their objective functions. For example, a problem that exhibits both multimodality and variable interactions is considered to combine two distinct features. This study focuses specifically on three key feature types: separable variables, interacting variables, and imbalanced variables, all of which are commonly used in evaluating the performance of MOEAs.

2.2. Reinforcement learning

Reinforcement learning (RL) serves as a fundamental machine learning paradigm [48], which has demonstrated remarkable success in various control applications ranging from the game of Go [49] to robot control [50]. In these applications, an agent learns a policy through interactions with its environment in order to maximize cumulative rewards. The agent's decision-making process is typically formulated as a Markov decision process (MDP), comprising:

- S: a set of states representing the environment and agent conditions;
- $\mathcal{A}$: a set of actions available to the agent;
- $P_a(s, s')$: the transition probability from state s to state s' under action a;
- $R_a(s, s')$: the immediate reward received after the transition from s to s' due to action a.

Fig. 1 shows an example of MDP, where one state s_t is changed to the next state s_{t+1} by taking action a_t based on policy $\pi : S \times \mathcal{A} \rightarrow [0, 1]$ accompanied by a feedback reward r_t. The trajectory of an MDP with T steps in dynamic environment is defined as the set $\tau = \{s_0, a_0, r_1, \ldots, a_{T-1}, r_T, s_T\}$. In an MDP, the intelligent RL agent aims to learn a policy $\pi(s, a)$ for maximizing the expected cumulative reward of the trajectory τ, i.e., $R(\tau) = \sum_{t=0}^{T-1} \gamma_t r_t$, where γ_t is a discount factor that prioritizes immediate rewards.

Q-learning is a model-free reinforcement learning algorithm [51] that does not require prior knowledge of the environment's transition dynamics. Instead, the agent learns through trial and error by interacting with the environment and updating its knowledge of the Q-values, the expected cumulative rewards for each pair of state-action combinations. These Q-values are stored in a Q-table, and the optimal policy is determined by selecting the action that maximizes the Q-value in each state. The Q-function is updated iteratively based on the Bellman equation:

$$Q(s_t, a_t) = (1 - \alpha) Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a \in \mathcal{A}} Q(s_{t+1}, a) \right] \tag{2}$$

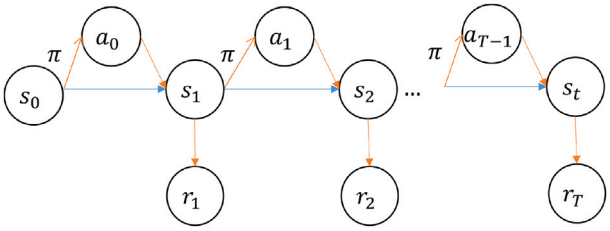


Fig. 1. An example of MDP in the dynamic environment with T steps.

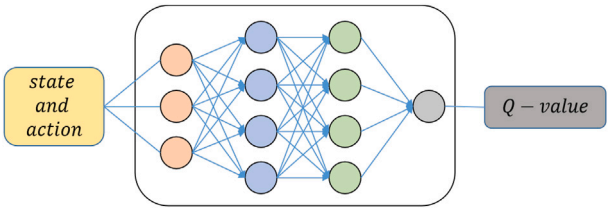


Fig. 2. DQN architecture diagram with state input (left), multi-layer neural network processing, and Q-value outputs (right) determining optimal action selection.

where α is the learning rate and γ is the discount factor that determines the importance of future rewards. Additionally, Q-learning balances exploration and exploitation using strategies like ϵ-greedy, where the agent randomly explores the environment with probability ϵ and exploits the best-known action with probability $1-\epsilon$.

The Deep Q-Network (DQN) [48] extends Q-learning by using deep neural networks to approximate the Q-function in high-dimensional state spaces. Its architecture employs two stabilization mechanisms: (1) Experience replay decouples sequential correlations through random sampling from a stored memory buffer, and (2) a periodically updated target network prevents policy oscillation. These innovations enable stable learning of generalized state-action mappings, allowing effective decision making in novel states within complex environments. The DQN architecture is shown in the Fig. 2.

2.3. Motivation

Although many researchers have achieved significant results with MOEAs, no single algorithm has yet demonstrated superiority across all types of problems. To overcome this limitation, this paper proposes a universal framework that integrates multiple MOEAs, enabling adaptive algorithm selection for solving diverse or mixed feature MOPs. Specifically, we incorporate three representative algorithms into the framework: NSGA-II, MOEA/D-DE, and MOEA/D-M2M. The rationale for selecting these algorithms is explained below:

- As emphasized in [17], NSGA-II has demonstrated remarkable success in solving continuous multi-objective benchmark problems such as ZDT1 through ZDT6. These problems feature separable variables in their distance functions, which result in Pareto optimal solutions that are mainly determined by the value of the first decision variable. This characteristic aligns well with NSGA-II's simulated binary crossover (SBX) operator, often leading to the production of high-quality offspring solutions.
- Unlike NSGA-II, the DE-enhanced MOEA/D variant (MOEA/D-DE [11,18]) is particularly advantageous for handling MOPs where variables interact within distance functions. Although it may exhibit slower convergence speeds compared to NSGA-II/SBX when tackling ZDT-type problems, MOEA/D-DE's approach is better suited for problems with complex variable interactions.
- MOEA/D-M2M [47] presents a novel approach for MOPs with imbalanced variables by decomposing them into a set of simpler multi-objective subproblems. This algorithm tackles these subproblems collaboratively, ensuring that each subproblem is equipped with its own population and receives the required computational resources at every generation. This strategy not only preserves population diversity but also effectively manages the complexities of MOPs with imbalanced variables.

In summary, each of the three algorithms works very well for a specific kind of problem: separable variables, interacting variables, or imbalanced variables. However, each algorithm performs well only on MOPs with its corresponding feature and poorly on other types. Since these features widely exist in real-world problems, it is beneficial to integrate algorithms that specialize in each feature. Recent studies [52,53] indicate a shift in control and optimization towards more automated methods, which calls for more self-adaptive algorithms. In response, we developed a novel RL framework that dynamically integrates multiple algorithms to address mixed-feature MOPs. Although the framework currently incorporates three algorithms for validation, it is designed to accommodate more in the future, thereby extending its applicability to a wider range of problems.

2.4. Proposed test problems

To validate the proposed approach and evaluate the algorithm's performance, we developed a new test suite (ZUM1-5) that systematically incorporates three distinct problem features. These features reflect the complexities present in real-world problems. For example, in aircraft wing optimization, parameters such as sweep, camber, and thickness exhibit variable interactions, while other variables remain separable. Furthermore, the introduction of imbalanced variables allows us to assess the algorithm's scalability and its ability to handle MOPs with diverse feature combinations. It is also important to note that, although the present study focuses on these three features, the framework is intentionally designed to be flexible and can readily be extended to incorporate additional problem features as needed.

The ZUM1–5 test problems are constructed to include three types of problem features: separable variables, interacting variables, and imbalanced variables. Their mathematical formulations are given as follows:

$$\begin{aligned} f_1(x) &= g \times \left(x_1 + \frac{2}{|J_1|} \sum_{j \in J_1} \left[x_j - \sin\left(6\pi x_1 + \frac{j\pi}{D}\right) \right]^2 \right) \\ f_2(x) &= \left(1 - \sqrt{\frac{x_1}{g}}\right) \times g \end{aligned} \tag{3}$$

where

- The search space is $[0,1]^{D_1+D_2} \times [-1,1]^{D-D_1-D_2}$, where $D_1 = \alpha \times D$ and $D_2 = \beta \times D$. Here, α and β are parameters that control the proportions of separable variables, imbalanced variables, and interacting variables. For example, when $\alpha = 0.3$ and $\beta = 0.1$, the separable variables account for 30 % of the decision variables, the imbalanced variables for 10 %, and the interacting variables for the remaining 60 %.
- The function g involved in f_2 is defined by:

$$\begin{aligned} g = 1 + \frac{9}{D_1 - 1} \sum_{i=2}^{D_1} x_i + \frac{2}{|J_2|} \sum_{j \in J_2} \left[x_j - \sin(6\pi x_1 + \frac{j\pi}{D}) \right]^2 \\ + \sin(\pi x_1) \sum_{j \in J_3} (-0.9 t_j^2 + |t_j|^{0.6}), \quad t_j = x_j - \sin(0.5\pi x_1) \end{aligned}$$

with $J_3 = \{j \in [D_1, D_1 + D_2]\}$, $J_1 = \{j \in [D_1 + D_2, D] | mod(j,2) = 1\}$ and $J_2 = \{j \in [D_1 + D_2, D] | mod(j,2) = 0\}$. The function g consists

of tree parts. The first part corresponds to ZDT-type problems with separable variables; the second part relates to UF-type problems with interacting variables; and the third part is derived from the MOP1–6 problems used for testing MOEA/D-M2M, which involve imbalanced variables.

The PF of (3) can be formulated as follows:

$$f_2 = 1 - \sqrt{f_1}, 0 \le f_1 \le 1.$$

The PS of (3) satisfies the following conditions:

$$0 \le x_1 \le 1; x_i = 0, i = 2, \dots, D';$$
$$x_j = \sin(0.5\pi x_1), j = D_1 + 1, \dots, D_1 + D_2$$
$$x_j = \sin\left(6\pi x_1 + j\frac{\pi}{D}\right), j = D_1 + D_2 + 1, \dots, D.$$

Based on Eq. (3), five test problems were considered, namely ZUM1 to ZUM5. The values of α, β in these test instances take the values in $\{(0.3, 0.4), (0.4, 0.5), (0.5, 0.6), (0.6, 0.7), (0.7, 0.8)\}$.

3. The description of RL-MOEA

As outlined in our research objectives, we propose an adaptive algorithm selection framework for solving various types of MOP, including those with mixed features. We present a novel reinforcement learning-based MOEA framework with dual operational modes: training mode and optimization mode. The framework employs an intelligent agent (RLA) that learns to dynamically switch between three core methods (NSGA-II, MOEA/D, and MOEA/D-M2M) during the search process, thereby enhancing the overall performance of the algorithm.

3.1. Main steps of RL-MOEA

The key steps of the RL-MOEA are outlined in Algorithm 1. The framework maintains two distinct populations:

- P_c: the current population, consisting of a set $x^1, \dots, x^N$ of N solutions used to generate offspring solutions;
- P_d: the diversity population, containing a set of N_d solutions used to reset P_c and maintain its diversity.

RL-MOEA operates in two modes: training mode, which runs for $l_{\max}$ loops (Lines 1–36) and involves training the agent; optimization mode, which executes a single loop (Lines 5–30) and uses the trained agent to solve MOPs.

- In training mode, the agent *RLA* is initialized in Lines 1–3 and updated using the trajectory $\tau = s_0, a_0, r_1, \dots, a_{T-1}, r_T, s_T$ in Line 31. Here, $T = \lfloor \frac{g_{\max}}{G} \rfloor$, meaning the entire evolutionary process is divided into T stages.
- In Line 5, the initial population P_c is generated via Latin hypercube sampling, while P_d is initialized as an empty set.
- In Line 6 and Line 19, under a given state s_t for $t = 1, \dots, T$, the action a_t is selected using the ϵ-greedy strategy described in Algorithm 2. During training, a random action is taken with 10 % probability for exploration, and the optimal action is chosen with 90 % probability for exploitation. In optimization mode, only optimal actions are selected based on the trained agent *RLA*. In Lines 9–15, depending on the selected action a_t, one of the three algorithms is used to update P_c.
- In Line 16, the diversity population P_d is updated according to Algorithm 3. First, all nondominated solutions in P_c are selected to form a set P_N. If P_d is empty, a solution x is randomly chosen from P_N and added to P_d. Then, the solution in P_N with the maximum distance to P_d is iteratively added until $|P_d| = N_d$. If P_d is not empty, the solution in P_N with the maximum distance to P_d is added only if this distance exceeds a threshold η; in this case, the oldest solution in P_d is removed to maintain the size N_d. The distance from a solution x to the set P_d is defined as:

$$\text{dist}(x, P_d) = \sum_{y \in P_d} \text{dist}(x, y) \tag{4}$$

where $\text{dist}(x, y)$ denotes the Euclidean distance between solutions x and y.
- In Lines 17–24, the next state s_{t+1} is computed, and the action a_{t+1} for the subsequent evolutionary stage is determined. If two consecutive

Algorithm 1: The framework of RL-MOEA.

input : *type* - the mode of training mode or optimization mode; N: the size of current population; N_d - the size of diverse population; RLA - the trained agent; g_{max} - the maximal number of generations, l_{max} - the maximal number of training iterations.
output: A agent RLA or a set of nondominated solutions

```
1  if type = training then
2  |  Initialize the parameters of agent RLA;
3  end
4  for l ← 1 to l_max do
5  |  Initialize P_c and P_d, calculate the initial state s_0;
6  |  a_0 ← ActionSelection(RLA, s_0, type);
7  |  Set g = 0 and t = 0 ;
8  |  while g ≤ g_max do
9  |  |  if a_t == 1 then
10 |  |  |  P_c ← NSGAII-SBX(P_c)
11 |  |  else if a_t == 2 then
12 |  |  |  P_c ← MOEA/D-DE(P_c)
13 |  |  else
14 |  |  |  P_c ← MOEA/D-M2M(P_c)
15 |  |  end
16 |  |  P_d ← UpdateDivPopu(P_c, P_d);
17 |  |  if g mod G = 0 then
18 |  |  |  s_{t+1} ← current state;
19 |  |  |  a_{t+1} ← ActionSelection(RLA,s_{t+1},type);
20 |  |  |  if a_t ≠ a_{t+1} then
21 |  |  |  |  P_c ← UpdateCurPopu(P_c, P_d, N_d);
22 |  |  |  end
23 |  |  |  a_t ← a_{t+1}, t ← t + 1;
24 |  |  end
25 |  |  g ← g + 1;
26 |  end
27 |  if type = optimization then
28 |  |  return nondominated solutions in P_c;
29 |  else
30 |  |  Update RLA using trajectory τ;
31 |  end
32 end
```

Algorithm 2: ActionSelection: the selection of action for a given state.

input : RLA - the agent; *state* - the input state; *type* - the model for optimization or training;
output: a - the selected action

```
1  if type = = 'training' then
2  |  a* = { argmax_{a∈A} RLA(state, a)   if rand() < 0.9
   |       { random action in A           otherwise
3  else
4  |  a* = argmax_{a∈A} RLA(state, a);
5  end
6  return a*
```

Algorithm 3: UpdateDivPopu: update diversity population.

input : P_c - the current population; P_d - the diversity population; N_d - the size of P_d.
output: P_d - the diversity population

Select all non dominated solutions from P_c to form P_N;
if $P_d == \emptyset$ **then**
  Randomly select a solution x from P_N and place it in P_d;
  while *size*$(P_d) < N_d$ **do**
    $P_d \leftarrow P_d \cup \{x : argmax_{x \in P_N} dist(x, P_d)\}$;
  end
else
  $x = argmax_{x \in P_N} dist(x, P_d)$;
  if $dist(x, P_d) > \eta$ **then**
    $P_d \leftarrow P_d \cup x$;
    Remove the oldest solution from P_d;
  else
    $P_d \leftarrow P_d$;
  end
end
return P_d

Algorithm 4: TrainRLA: train agent RLA.

input : τ - a set of trajectory; $training_{method}$ - training approach (Q-learning or DQN); T - number of stages; RLA - agent before training
output: RLA - trained agent

1 **if** $training_{method} ==' Q-learning'$ **then**
2 **for** $t = 1 \rightarrow T$ **do**
3 **if** $t < T$ **then**
4 $Q(s_t, a_t) = (1-\alpha)Q(s_t, a_t) + \alpha[r_{t+1} + \gamma \max_{a \in \mathcal{A}} Q(s_{t+1}, a)]$;
5 **else**
6 $Q(s_t, a_t) = (1-\alpha)Q(s_t, a_t) + \alpha r_{t+1}$;
7 **end**
8 **end**
9 $RLA = Q(s, a)$;
10 **else**
11 $w^* = 0$;
12 **for** $t = 1 \rightarrow T-1$ **do**
13 $r_t = r_t + \gamma \max_{a \in \mathcal{A}} DQN(s_{t+1}, a_{t+1}, w)$;
14 $w^* = w^* + \nabla_w ||r_t - DQN(s_t, a_t, w)||^2$;
15 **end**
16 $w^* = w^* + \nabla_w ||r_T - DQN(s_T, a_T, w)||^2$;
17 $w = w - \alpha w^*$;
18 $RLA = DQN(s, a, w)$;
19 **end**
20 return RLA

actions differ, i.e., $a_t \neq a_{t+1}$, the current population P_c is reset using solutions from P_d: the best $N - N_d$ solutions in P_c are retained, and the rest are replaced with solutions from P_d.

- In Lines 27–31, if the framework is in optimization mode, all nondominated solutions are directly output. In training mode, the reward is computed first, and the agent *RLA* is updated according to Algorithm 4.

3.2. Training the agent RLA

In this paper, the evolution process of RL-MOEA is modeled as a Markov Decision Process (MDP), and the agent RLA is trained using Q-learning or DQN algorithms. As outlined in Algorithm 1, a trajectory set $\tau = s_0, a_0, r_1, \ldots, a_{T-1}, r_T, s_T$ is generated. Additionally, we referred to the reward formulation proposed in [54]. The definitions of states, actions, and rewards within the RL-MOEA framework are as follows:

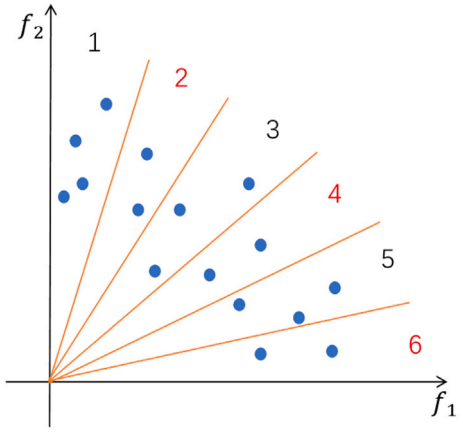


Fig. 3. The target space is uniformly divided into K regions. s_t^2 represents the number of regions where the number of solutions is lower than γ.

- The state of population P_c at time t is represented as a pair (s_t^1, s_t^2), which quantifies the convergence and diversity quality of P_c, respectively.
  - The value of s_t^1 is computed as the average objective function value of all solutions in P_c:

$$s_t^1 = \frac{1}{N \times m} \sum_{i=1}^{N} \sum_{j=1}^{m} f_j(x^i) \tag{5}$$

  - The value of s_t^2 is obtained by uniformly dividing the objective space into K regions and counting how many regions contain fewer than γ solutions. For example, as shown in Fig. 3, if the space is divided into six regions and $\gamma = 3$, and regions 2, 4, and 6 have fewer than three solutions, then $s_t^2 = 3$. This approach eliminates the need for discretization of s_t^2 when using Q-learning.
- The action space is defined as $\mathcal{A} = 1, 2, 3$, where each action corresponds to optimizing P_c with a specific algorithm: $a = 1$ uses NSGA-II with SBX, $a = 2$ uses MOEA/D-DE, and $a = 3$ uses MOEA/D-M2M.
- The reward r_k is set to 0 for $k = 0, \ldots, T-1$, while the terminal reward r_T is assigned the hypervolume value of P_c.

Algorithm 4 describes the procedure for training the agent RLA using the trajectory set τ.

- Lines 1–9 implement Q-learning to train a Q-table. Since Q-learning requires discrete state-action pairs, s_t^1 is discretized into bins: $\{[0, 0.45), [0.45, 0.5), [0.5, 0.55), [0.55, 0.6), [0.6, 0.65), [0.65, +\infty)\}$.
- In lines 11–17, a neural network is trained using the DQN algorithm. The inputs are state-action pairs, and the output is the corresponding Q-value. Here, w denotes the weights of the neural network.

Once the Q-table or DQN is trained, the optimal action is selected by maximizing the Q-value for the current state. Note that if the agent is being trained for the first time, its parameters must be initialized. In this work, Q-learning initializes all $Q(s, a)$ values to 0, while DQN initializes the weights w randomly.

4. Computational experiments

This section first introduces the performance indicators used to evaluate RL-MOEA. Subsequently, we employ widely used test suites, namely UF1-UF7 [12], ZDT1-4, and ZDT6 [8], as well as MOP1-6 [47], to

compare RL-MOEA with three baseline MOEAs: NSGA-II, MOEA/D-DE, and MOEA/D-M2M. Finally, we provide a comparative analysis of two alternative training algorithms for the RL agent.

Each algorithm was independently run 20 times on each test problem. To statistically compare the performance of RL-MOEA against the baseline algorithms, the Wilcoxon rank–sum test was applied with a significance level of 0.05. The symbols "+", "−", and "≈" indicate that RL-MOEA performs significantly better than, significantly worse than, or is statistically equivalent to the compared algorithm, respectively. All the compared algorithms were implemented in PlatEMO [55].

4.1. Performance indicators

In the experiments, two widely used performance indicators, the inverse generational distance (IGD) and hypervolume value(HV), were used to assess the performance of the compared algorithms.

- **Inverted Generational Distance (IGD)**: This metric measures the average distance from each point in the true Pareto front R to the nearest solution in the approximate set A obtained by the algorithm. It is defined as:

$$IGD(A, R) = \frac{1}{|R|} \sum_{u \in R} \min_{v \in A} \text{dist}(u, v)$$

  where $\text{dist}(u, v)$ represents the Euclidean distance between points u and v.
- Hypervolume (HV): This metric calculates the volume of the space dominated by the solution set A with respect to a predefined reference point $r^* = (r_1^*, \ldots, r_m^*)$. The HV is computed using the following formula:

$$HV(A, r^*) = \Lambda\left(\bigcup_{u \in A} [u_1, r_1^*] \times \cdots \times [u_m, r_m^*]\right)$$

  where $\Lambda(\cdot)$ denotes the Lebesgue measure of the union of dominated hypercubes in the objective space.

4.2. General parameter settings

To verify the performance of RL-MOEA, three other baseline MOEAs, i.e., NSGA-II/SBX, MOEA/D-SBX, and MOEA/D-DE, are considered in our experiments. In all four algorithms, the population size is set to 100. The following parameter settings are used in RL-MOEA:

- The learning rate α and the discount factor γ are set to 0.1 and 0.9 respectively;
- The number of weight vectors K used in calculating the value of diversity state s_t^2 is set to 6;
- The number of generations in each search stage, denoted as G, is set to $g_{\max}/10$;

In addition, the neighborhood size in MOEA/D is set to $0.2 \times N$. The distribution indices for simulated binary crossover (SBX) and polynomial mutation are both set to 20. The scaling factor F and crossover rate CR in the differential evolution (DE) operator are set to 0.5 and 1.0, respectively.

The training process for RL-MOEA agents utilizes the DQN algorithm and is carried out over a maximum of 300 generations. All training is performed exclusively on the ZUM2 benchmark problem, which contains 20 decision variables. Notably, the agent only requires training once on a single problem. After training, it can be applied directly to solve new optimization problems without any further training.

4.3. Performance on various benchmarks

This section evaluates the performance of the four algorithms on two-objective benchmark test problems, namely UF1–UF7, ZDT1–4, ZDT6, and MOP1–6. The maximum number of generations is set to 100 in all scenarios.

Table 1 presents the performance of the four algorithms across seventeen test instances, revealing several key trends and observations:

- Among the four algorithms, RL-MOEA achieved the lowest IGD value in 7 test instances. Furthermore, RL-MOEA outperformed MOEA/D-DE in 12 test instances, MOEA/D-M2M in 4 test instances, and NSGA-II in 12 test instances. These results demonstrate the effectiveness of RL-MOEA in multi-objective optimization.
- Overall, RL-MOEA significantly outperformed MOEA/D-DE and MOEA/D-M2M, while it was slightly outperformed by NSGA-II in a few instances. Specifically, on the UF problems, RL-MOEA showed a clear advantage over NSGA-II, while performing comparably to MOEA/D-DE and MOEA/D-M2M. On the MOP1–6 problems, RL-MOEA exhibited notable improvements over MOEA/D-DE and NSGA-II, while maintaining performance comparable to MOEA/D-M2M.

In summary, although RL-MOEA may not achieve optimal performance for certain types of test problems, it demonstrates strong generalizability and can effectively tackle a diverse range of MOPs. In particular, when applied to problems with unknown features, RL-MOEA proves to be both reliable and adaptable, making it a highly suitable choice for practical applications.

Table 1
The mean and standard deviation(in parentheses) of IGD values obtained by four algorithms on Benchmark Problems. The minimal mean IGD value is highlighted in bold for each test instance.

Problem	D	MOEADDE	MOEADM2M	NSGAII	RL-MOEA
ZDT1	20	1.3679e−1 (6.14e−2) –	1.2102e−2 (7.62e−3) ≈	**6.4782e−3 (6.84e−4)** +	8.9340e−3 (2.10e−3)
ZDT2	20	2.4549e−1 (2.16e−1) –	9.8251e−3 (3.04e−3) –	9.3502e−3 (1.15e−2) –	**8.0012e−3 (1.77e−3)**
ZDT3	20	1.8444e−1 (5.35e−2) –	6.0557e−2 (3.72e−2) –	**1.0474e−2 (1.07e−2)** +	4.2287e−2 (3.96e−2)
ZDT4	20	4.2888e+0 (1.49e+0) –	4.2694e+1 (1.12e+1) –	**6.6197e−2 (5.12e−2)** +	1.3160e+0 (8.26e−1)
ZDT6	20	2.0358e+0 (8.73e−1) –	9.2691e−2 (1.26e−1) –	7.2991e−1 (1.40e−1) –	**1.7923e−2 (2.45e−2)**
UF1	10	**8.9486e−3 (1.59e−2)** +	1.5296e−2 (3.02e−3) ≈	4.6505e−2 (1.14e−2) –	1.3624e−2 (2.43e−3)
UF2	10	**1.1214e−2 (2.26e−3)** +	1.1516e−2 (1.06e−3) ≈	2.1213e−2 (5.84e−3) –	1.1862e−2 (6.28e−4)
UF3	10	1.4845e−1 (7.05e−2) ≈	**9.9444e−2 (3.96e−2)** ≈	2.2214e−1 (4.56e−2) –	1.0540e−1 (3.91e−2)
UF4	10	5.1971e−2 (4.46e−3) –	4.3481e−2 (1.20e−3) ≈	**4.0269e−2 (1.99e−3)** +	4.3771e−2 (1.20e−3)
UF5	10	3.0036e−1 (9.52e−2) –	1.7087e−1 (3.83e−2) ≈	2.6624e−1 (5.10e−2) –	**1.6341e−1 (2.65e−2)**
UF6	10	2.9469e−1 (2.02e−1) ≈	2.4019e−1 (7.41e−2) ≈	**2.0514e−1 (8.69e−2)** ≈	2.0644e−1 (7.70e−2)
UF7	10	**6.3528e−3 (8.11e−4)** +	1.1791e−2 (1.56e−3) ≈	7.0645e−2 (9.39e−2) –	1.1341e−2 (1.21e−3)
MOP1	10	3.0266e−1 (7.52e−2) –	**5.7364e−2 (6.42e−3)** ≈	3.5046e−1 (5.36e−3) –	5.9463e−2 (9.52e−3)
MOP2	10	2.8921e−1 (5.35e−2) –	2.0991e−1 (5.93e−2) ≈	3.5494e−1 (1.11e−16) –	**2.0323e−1 (6.40e−2)**
MOP3	10	5.4597e−1 (4.64e−2) –	3.4185e−1 (1.16e−1) ≈	5.7216e−1 (5.41e−2) –	**2.9175e−1 (9.85e−2)**
MOP4	10	2.6100e−1 (2.63e−2) –	1.3392e−1 (7.69e−2) ≈	2.9415e−1 (1.65e−2) –	**1.2994e−1 (7.11e−2)**
MOP5	10	3.1583e−1 (9.64e−3) –	1.0344e−1 (2.62e−2) ≈	2.8677e−1 (3.06e−2) –	**9.4075e−2 (1.98e−2)**
+/−/≈		3/12/2	0/4/13	4/12/1	

Table 2
The mean and standard deviation(in the parentheses) of IGD values obtained by four algorithms on two-objective ZUM1-ZUM5. The minimal mean IGD value is highlighted in bold for each test instance.

Problem	D	MOEADDE	MOEADM2M	NSGAII	RL-MOEA
ZUM1	10	9.2385e−2 (1.06e−1) ≈	6.5019e−2 (1.30e−2) ≈	1.0353e−1 (3.05e−2) –	**5.9886e−2 (1.30e−2)**
	20	1.0415e−1 (2.87e−2) –	9.2236e−2 (2.51e−2) ≈	1.3951e−1 (3.98e−2) –	**7.9022e−2 (1.18e−2)**
ZUM2	10	7.1097e−2 (5.11e−2) ≈	5.7116e−2 (1.40e−2) ≈	1.0530e−1 (4.03e−2) –	**5.6450e−2 (1.09e−2)**
	20	9.5225e−2 (1.78e−2) –	8.2536e−2 (1.79e−2) ≈	1.5538e−1 (4.03e−2) –	**8.0588e−2 (1.40e−2)**
ZUM3	10	6.1134e−2 (1.70e−2) –	5.7637e−2 (1.34e−2) –	9.5784e−2 (3.07e−2) –	**4.8626e−2 (1.01e−2)**
	20	1.0425e−1 (5.04e−2) –	8.9756e−2 (1.63e−2) –	1.1673e−1 (3.01e−2) –	**7.3989e−2 (1.10e−2)**
ZUM4	10	5.1250e−2 (2.04e−2) ≈	4.7853e−2 (1.42e−2) ≈	9.0579e−2 (3.45e−2) –	**4.4493e−2 (1.29e−2)**
	20	9.0319e−2 (2.99e−2) –	7.6309e−2 (1.48e−2) –	1.2844e−1 (6.27e−2) –	**6.5522e−2 (1.09e−2)**
ZUM5	10	4.3910e−2 (2.07e−2) ≈	4.3735e−2 (1.25e−2) ≈	6.7736e−2 (1.43e−2) –	**3.8383e−2 (8.04e−3)**
	20	7.5900e−2 (3.02e−2) –	6.8282e−2 (1.20e−2) –	1.1549e−1 (3.54e−2) –	**5.0958e−2 (8.03e−3)**
+/−/≈		0/6/4	0/4/6	0/10/0	

Table 3
The mean and standard deviation(in the parentheses) of HV values obtained by four algorithms on two-objective ZUM1-ZUM5. The maximal mean HV value is highlighted in bold for each test instance.

Problem	D	MOEADDE	MOEADM2M	NSGAII	RL-MOEA
ZUM1	10	6.0750e−1 (1.15e−1) ≈	6.3159e−1 (1.79e−2) ≈	5.8961e−1 (4.38e−2) –	**6.4240e−1 (1.80e−2)**
	20	5.7581e−1 (4.26e−2) –	5.9163e−1 (3.32e−2) –	5.5149e−1 (4.91e−2) –	**6.1241e−1 (1.74e−2)**
ZUM2	10	6.3175e−1 (6.34e−2) ≈	6.4368e−1 (1.93e−2) ≈	5.9265e−1 (5.55e−2) –	**6.4455e−1 (1.57e−2)**
	20	5.8883e−1 (2.66e−2) –	6.0560e−1 (2.45e−2) ≈	5.2486e−1 (5.16e−2) –	**6.1156e−1 (1.93e−2)**
ZUM3	10	6.4471e−1 (1.82e−2) ≈	6.4277e−1 (1.68e−2) ≈	6.0819e−1 (3.57e−2) –	**6.5365e−1 (1.52e−2)**
	20	5.8092e−1 (7.13e−2) –	5.9713e−1 (1.82e−2) –	5.7557e−1 (4.25e−2) –	**6.2193e−1 (1.61e−2)**
ZUM4	10	6.5800e−1 (2.59e−2) ≈	6.5485e−1 (2.02e−2) ≈	6.1907e−1 (3.61e−2) –	**6.6306e−1 (1.58e−2)**
	20	6.0618e−1 (3.96e−2) –	6.1604e−1 (1.93e−2) –	5.6200e−1 (8.85e−2) –	**6.3853e−1 (1.32e−2)**
ZUM5	10	6.6755e−1 (2.34e−2) ≈	6.6439e−1 (1.56e−2) ≈	6.4518e−1 (1.70e−2) –	**6.7095e−1 (1.14e−2)**
	20	6.2436e−1 (3.96e−2) –	6.2712e−1 (1.34e−2) –	5.8414e−1 (4.06e−2) –	**6.5603e−1 (1.06e−2)**
+/−/≈		0/5/5	0/4/6	0/10/0	

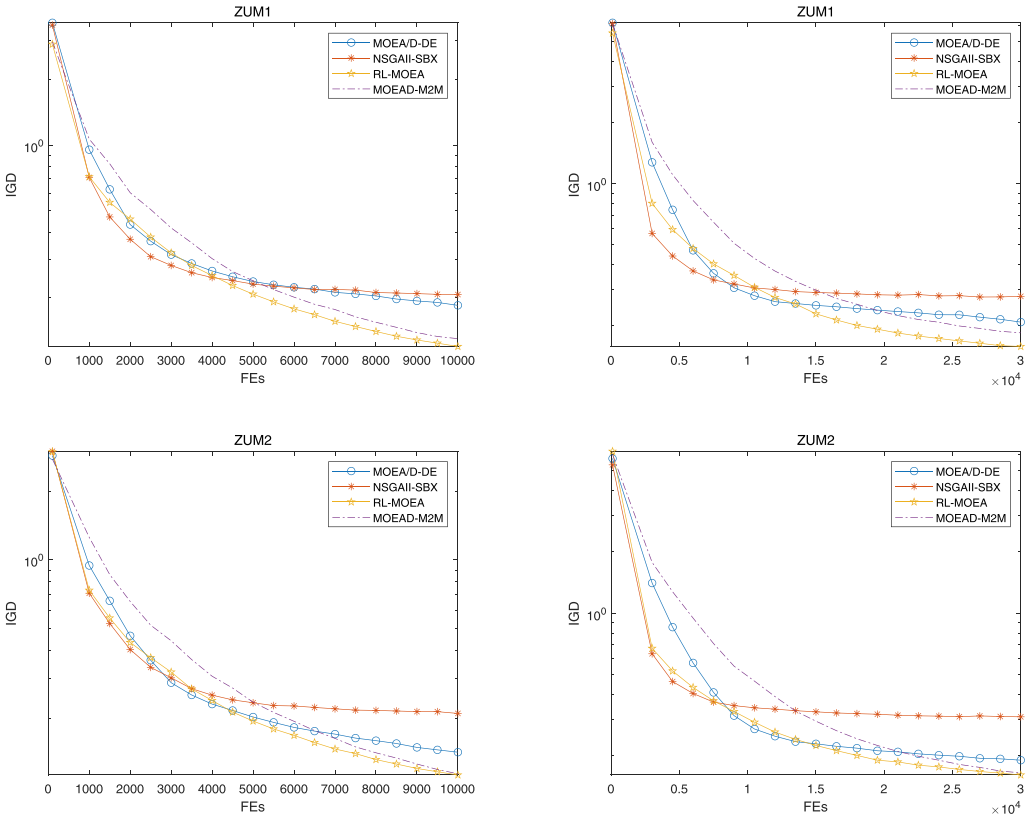


Fig. 4. The evolution of the mean IGD values found by four compared algorithms on ZUM1-2 with 10 and 20 decision variables.

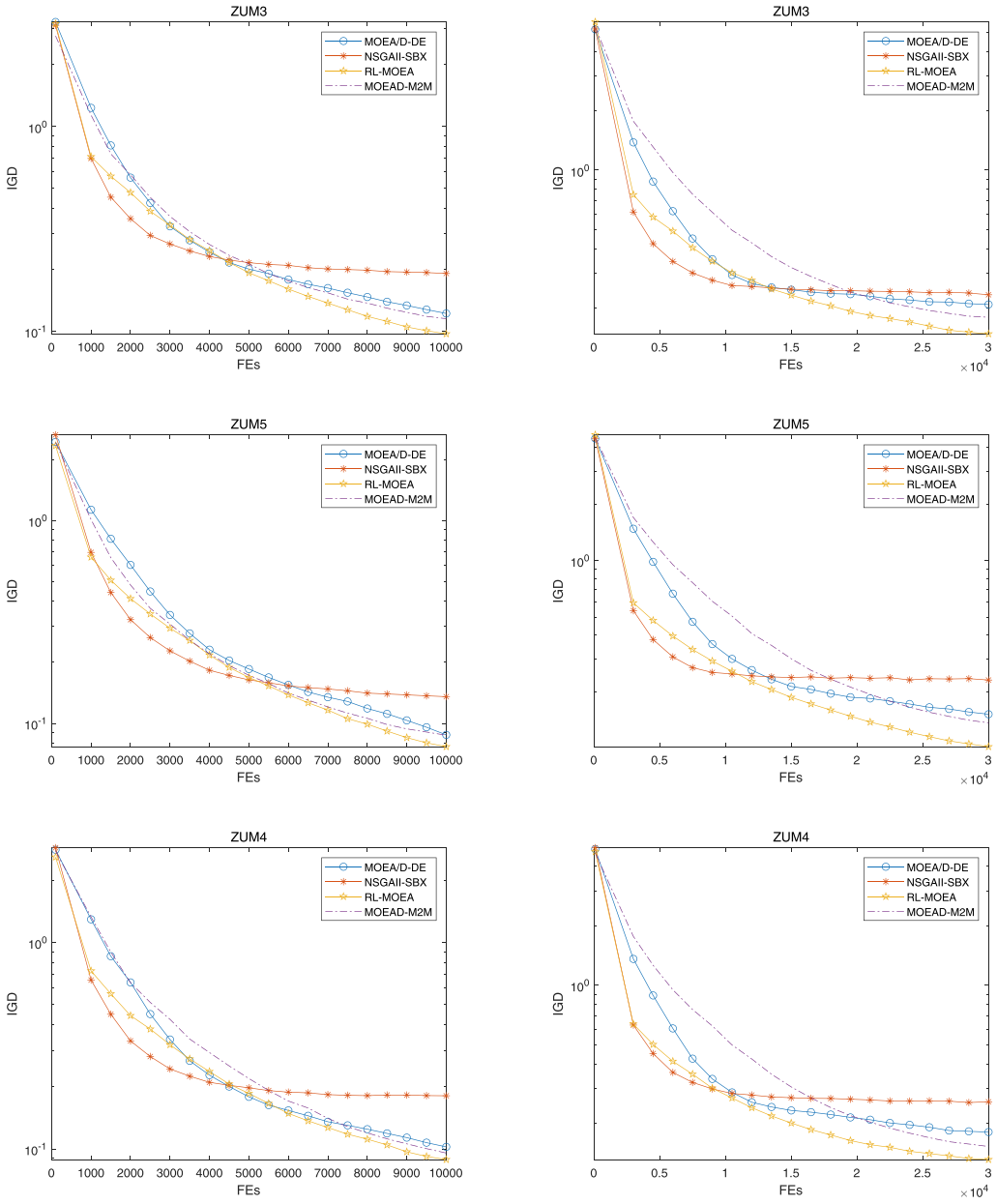


Fig. 5. The evolution of the mean IGD values found by four compared algorithms on ZUM3-5 with 10 and 20 decision variables.

Table 4
The mean and standard deviation(in parentheses) of IGD values obtained by two algorithms on two-objective ZUM1-ZUM5. The minimal mean IGD value is highlighted in bold for each test instance.

Problem	D	RL-MOEA-QL	RL-MOEA-DQN
ZUM1	10	**5.5798e−2 (1.90e−2)** ≈	5.9886e−2 (1.30e−2)
	20	8.4020e−2 (1.41e−2) ≈	**7.9022e−2 (1.18e−2)**
ZUM2	10	**4.7773e−2 (1.32e−2)** –	5.6450e−2 (1.09e−2)
	20	8.2434e−2 (1.38e−2) ≈	**8.0588e−2 (1.40e−2)**
ZUM3	10	5.2291e−2 (1.66e−2) ≈	**4.8626e−2 (1.01e−2)**
	20	7.7998e−2 (2.96e−2) ≈	**7.3989e−2 (1.10e−2)**
ZUM4	10	5.1572e−2 (1.83e−2) ≈	**4.4493e−2 (1.29e−2)**
	20	**6.4750e−2 (1.35e−2)** ≈	6.5522e−2 (1.09e−2)
ZUM5	10	**3.4930e−2 (1.35e−2)** ≈	3.8383e−2 (8.04e−3)
	20	5.2651e−2 (1.81e−2) ≈	**5.0958e−2 (8.03e−3)**
+/−/≈		1/0/9	

4.4. Performance on ZUM

This section evaluates the performance of four algorithms on the ZUM test suite, which includes problems with 10 and 20 decision variables.

Tables 2 and 3 present the average IGD and HV values obtained by the four algorithms on each test instance. The results clearly show that RL-MOEA outperforms the other algorithms in most cases. Specifically, RL-MOEA consistently achieves the lowest average IGD values across all ten instances. Moreover, the HV values obtained by RL-MOEA are significantly better than those of the other three algorithms, demonstrating its superior performance and effectiveness in solving MOPs with mixed features.

Notably, RL-MOEA consistently performs better than NSGA-II in every instance. For the 10-dimensional problems, MOEA/D-M2M and MOEA/D-DE exhibit performance levels similar to RL-MOEA. However,

Table 5
The mean and standard deviation(in parentheses) of IGD values obtained by two algorithms on Benchmark Problems. The minimal mean IGD value is highlighted in bold for each test instance.

Problem	D	RL-MOEA-QL	RL-MOEA-DQN
ZDT1	20	2.2850e−2 (1.14e−2) –	**8.9340e−3 (2.10e−3)**
ZDT2	20	1.5025e−2 (1.29e−2) ≈	**8.0012e−3 (1.77e−3)**
ZDT3	20	**2.9899e−2 (1.41e−2)** ≈	4.2287e−2 (3.96e−2)
ZDT4	20	1.4694e+1 (7.46e+0) –	**1.3160e+0 (8.26e−1)**
ZDT6	20	1.3846e−1 (1.55e−1) –	**1.7923e−2 (2.45e−2)**
UF1	10	**1.3157e−2 (3.13e−3)** ≈	1.3624e−2 (2.43e−3)
UF2	10	**1.1449e−2 (9.65e−4)** ≈	1.1862e−2 (6.28e−4)
UF3	10	1.0700e−1 (5.62e−2) ≈	**1.0540e−1 (3.91e−2)**
UF4	10	**4.1673e−2 (1.61e−3) –**	4.3947e−2 (1.54e−3)
UF5	10	2.0799e−1 (6.37e−2) –	**1.6341e−1 (2.65e−2)**
UF6	10	2.1613e−1 (7.26e−2) ≈	**2.0644e−1 (7.70e−2)**
UF7	10	2.2083e−2 (6.35e−2) –	**1.1341e−2 (1.21e−3)**
MOP1	10	1.4073e−1 (1.08e−1) –	**5.9463e−2 (9.52e−3)**
MOP2	10	2.3818e−1 (4.94e−2) ≈	**2.0323e−1 (6.40e−2)**
MOP3	10	4.9666e−1 (7.33e−2) –	**2.9175e−1 (9.85e−2)**
MOP4	10	2.3900e−1 (1.55e−2) –	**1.2994e−1 (7.11e−2)**
MOP5	10	3.0155e−1 (3.83e−2) –	**9.4075e−2 (1.98e−2)**
+/−/≈		1/9/6	

as the dimensionality increases to 20, the performance of all algorithms deteriorates. Despite this, RL-MOEA maintains an advantage over both MOEA/D-M2M and MOEA/D-DE, highlighting its stronger optimization capability and stability in higher-dimensional spaces.

Figs. 4 and 5 show the evolution of the IGD values obtained by the four algorithms on ZUM1–5 with 10 and 20 decision variables, respectively. The following observations can be drawn:

- RL-MOEA demonstrates the fastest convergence across all ten test instances. In addition, both RL-MOEA and NSGA-II/SBX converge more rapidly than the other two algorithms during the early stages of evolution.
- As the evolution proceeds to the mid-stage, the convergence speed of NSGA-II/SBX and MOEA/D-DE decreases or stagnates. In contrast, RL-MOEA continues to consistently find better solutions.
- It is evident that only RL-MOEA maintains the fastest global convergence speed. This indicates that the trained agent can effectively select the most suitable algorithm at different stages of the evolution process.

In summary, RL-MOEA exhibits excellent performance on the newly designed ZUM test suite, confirming its effectiveness in obtaining high-quality solutions. Notably, as the problem dimensionality increases, RL-MOEA outperforms the other algorithms by identifying higher-quality solutions more efficiently.

4.5. Comparison of different agents

This paper introduces two reinforcement learning algorithms for agent training: Q-learning and DQN, referred to as RL-MOEA-QL and RL-MOEA-DQN, respectively. Both methods are trained on the ZUM2 problem with 20 decision variables.

Table 4 presents the IGD values obtained by the two algorithms on the ZUM problems with 10 and 20 decision variables. Overall, the performance of both algorithms is comparable; however, RL-MOEA-DQN exhibits slightly inferior performance compared to RL-MOEA-QL on the ZUM2 problem with 10 decision variables.

Table 5 shows the IGD values of both algorithms on two-objective benchmark test problems. Among the seventeen test instances, RL-MOEA-DQN outperforms RL-MOEA-QL in nine instances, performs comparably in six, and underperforms in only one.

In summary, while RL-MOEA-DQN slightly underperforms RL-MOEA-QL on the ZUM test problems, it demonstrates significantly stronger generalization capability and adaptability across a broader range of benchmark problems. This suggests that RL-MOEA-DQN is more general and potentially better suited for solving a wider variety of MOPs. This advantage may be attributed to the enhanced stability of DQN over Q-learning, allowing it to manage larger action and state spaces more effectively.

4.6. Sensitivity analysis

In our proposed RL-MOEA, the major parameter N_d, which controls the size of the diversity population, could affect the performance of RL-MOEA on convergence and diversity. A larger value of N_d may slow down the convergence speed, while a smaller value of N_d may cause the reduction of diversity. Therefore, selecting an appropriate value for N_d is essential to balance convergence and diversity, especially for MOPs with mixed features.

To analyze the sensitivity of this parameter, we tested RL-MOEA on the ZUM1-5 test suite with 10 and 20 decision variables. We evaluated five values of N_d: 10, 20, 30, 40, and 50, with corresponding variants named RL-MOEA-10 to RL-MOEA-50.

Table 6 presents the statistical IGD results under these settings. The five algorithms achieved similar performance, with RL-MOEA-30 performing the best and RL-MOEA-10 performing the worst. Based on these results, we set $N_d = 30$ in subsequent experiments to ensure stable and balanced performance. Overall, RL-MOEA shows low sensitivity to the setting of N_d, demonstrating good robustness.

4.7. Complexity analysis

The proposed framework dynamically integrates three multi-objective evolutionary algorithms: NSGA-II, MOEA/D, and MOEA/D-M2M. As a result, its computational complexity depends on which algorithm is active during a given evolutionary phase.

The time complexity of NSGA-II is $O(M \times N^2)$, while MOEA/D has a complexity of $O(M \times N \times T)$, where T is the neighborhood size. MOEA/D-M2M has a complexity of $O(M \times N^2/K)$, with K representing the number of subpopulations. Accordingly, the overall time complexity of the framework ranges between $O(M \times N \times T)$, $O(M \times N^2/K)$, and $O(M \times N^2)$, depending on the algorithm in use and its configuration.

Table 6
The mean and standard deviation (in parentheses) of IGD values obtained by five algorithms on two-objective ZUM1-ZUM5. The minimal mean IGD value is highlighted in bold for each test instance.

Problem	D	RL-MOEA-10	RL-MOEA-20	RL-MOEA-30	RL-MOEA-40	RL-MOEA-50
ZUM1	10	6.0746e−2 (7.91e−3) ≈	6.0281e−2 (9.88e−3) ≈	5.7109e−2 (1.13e−2) ≈	**5.4966e−2** (1.29e−2) +	6.1993e−2 (1.21e−2)
	20	8.7556e−2 (1.88e−2) ≈	8.1924e−2 (1.94e−2) ≈	8.7032e−2 (1.88e−2) ≈	8.7040e−2 (1.25e−2) –	**7.9678e−2** (1.42e−2)
ZUM2	10	5.6309e−2 (1.45e−2) ≈	**5.3140e−2** (1.22e−2) ≈	5.4051e−2 (1.08e−2) ≈	5.9045e−2 (1.49e−2) ≈	5.7226e−2 (1.16e−2)
	20	8.5171e−2 (1.13e−2) ≈	8.5814e−2 (1.19e−2) ≈	**7.8545e−2** (1.28e−2) ≈	8.0952e−2 (9.66e−3) ≈	8.4142e−2 (1.57e−2)
ZUM3	10	5.4757e−2 (1.37e−2) ≈	5.8166e−2 (1.47e−2) –	5.3080e−2 (1.09e−2) ≈	5.1673e−2 (1.56e−2) ≈	**4.9410e−2** (1.12e−2)
	20	7.6635e−2 (1.11e−2) ≈	8.5383e−2 (1.62e−2) ≈	8.1986e−2 (1.90e−2) ≈	**7.5000e−2** (1.24e−2) ≈	8.0888e−2 (1.32e−2)
ZUM4	10	**4.2657e−2** (8.12e−3) ≈	4.8254e−2 (1.10e−2) ≈	4.5448e−2 (1.26e−2) ≈	4.5284e−2 (1.15e−2) ≈	4.4234e−2 (1.40e−2)
	20	6.6377e−2 (1.50e−2) ≈	7.1810e−2 (1.82e−2) ≈	**6.4513e−2** (1.02e−2) ≈	6.9499e−2 (1.61e−2) ≈	7.0045e−2 (1.50e−2)
ZUM5	10	3.9724e−2 (1.01e−2) ≈	**3.7564e−2** (9.38e−3) ≈	3.9927e−2 (1.34e−2) ≈	3.9256e−2 (9.85e−3) ≈	3.8105e−2 (9.89e−3)
	20	5.2216e−2 (1.00e−2) ≈	5.4074e−2 (7.78e−3) ≈	**5.0571e−2** (5.95e−3) ≈	5.2739e−2 (8.14e−3) ≈	5.1390e−2 (1.08e−2)
+/−/≈		0/0/10	0/1/9	0/0/10	1/1/8	

The DQN agent, which includes L layers with U units per layer, introduces an additional per-step complexity of $O(L \times U^2)$. However, since the DQN is trained offline, this does not affect online performance. Memory overhead is mainly attributed to the experience replay buffer and remains manageable.

As a result, the online optimization phase remains efficient, with computational cost dominated by the selected MOEA. This makes the framework suitable for real-time applications.

5. Conclusions

This paper proposes the RL-MOEA framework, which integrates multiple MOEAs to significantly improve generalization performance. This enables RL-MOEA to handle various types of MOPs, including those with mixed features. By modeling the population evolution process as an MDP and using Q-learning or DQN algorithms to train intelligent agents, RL-MOEA dynamically adjusts its optimal method at different evolutionary stages, thereby adapting to the specific features of MOPs. Experimental results show that RL-MOEA consistently obtains high-quality solutions more efficiently than the individual embedded MOEAs on the ZUM test suite, especially as the problem dimensionality increases. Furthermore, although RL-MOEA achieves competitive performance across various benchmarks, it does not outperform every embedded algorithm in every specific test instance.

In summary, RL-MOEA integrates with various MOEAs to form a general and effective framework for solving diverse complex MOPs. By leveraging trained reinforcement learning agents, it enhances robustness across different application scenarios and demonstrates strong potential in both algorithm selection and problem-specific adaptation. The method has been validated on benchmark problems as well as mixed-variable MOPs commonly found in real-world applications such as aircraft wing design. Future work will focus on applying RL-MOEA to practical domains like engineering design and economic modeling, which will require integration with domain-specific knowledge and constraints, thereby offering further insight into its scalability and effectiveness.

CRediT authorship contribution statement

Yanhui Tang: Writing – original draft, Visualization, Software, Methodology, Investigation, Data curation, Conceptualization. **Hui Li:** Writing – review & editing, Project administration, Methodology, Funding acquisition, Conceptualization. **Haotian Zhang:** Software, Methodology, Conceptualization. **Jianyong Sun:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partly supported by the National Natural Science Foundation of China (grant no. 62072364).

Data availability

The authors do not have permission to share data.

References

[1] J. Branke, B. Scheckenbach, M. Stein, K. Deb, H. Schmeck, Portfolio optimization with an envelope-based multi-objective evolutionary algorithm, Eur. J. Oper. Res. 199 (3) (2009) 684–693.
[2] X. Zhang, Y. Zhou, Q. Zhang, V.C.S. Lee, M. Li, Problem specific MOEA/d for barrier coverage with wireless sensors, IEEE Trans. Cybern. 47 (11) (2017) 3854–3865.
[3] N. Jozefowiez, F. Semet, E.-G. Talbi, Multi-objective vehicle routing problems, Eur. J. Oper. Res. 189 (2) (2008) 293–309.
[4] H.S. Kim, C. Park, V. Govindan, S. Vasudevan, K.G. Jagan, Systematic reliability optimization (ASRO), Babyl. J. Math. (2023) 50–53.
[5] K. Miettinen, Nonlinear Multiobjective Optimization, Springer Science & Business Media, 1999.
[6] R.T. Marler, J.S. Arora, Survey of multi-objective optimization methods for engineering, Struct. Multidiscip. Optim. 26 (2004) 369–395.
[7] G. Mavrotas, Effective implementation of the ε-constraint method in multi-objective mathematical programming problems, Appl. Math. Comput. 213 (2) (2009) 455–465.
[8] E. Zitzler, K. Deb, L. Thiele, Comparison of multiobjective evolutionary algorithms: empirical results, Evol. Comput. 8 (2) (2000) 173–195.
[9] K. Deb, L. Thiele, M. Laumanns, E. Zitzler, Scalable multi-objective optimization test problems, in: Proceedings of the 2002 Congress on Evolutionary Computation, vol. 1, 2002, pp. 825–830.
[10] V.L. Huang, A.K. Qin, K. Deb, E. Zitzler, P.N. Suganthan, J.J. Liang, M. Preuss, S. Huband, Problem definitions for performance assessment of multi-objective optimization algorithms 10 (2007).
[11] H. Li, Q. Zhang, Multiobjective optimization problems with complicated pareto sets, MOEA/d and NSGA-II, IEEE Trans. Evol. Comput. 13 (2) (2009) 284–302.
[12] Q. Zhang, A. Zhou, S. Zhao, P. Suganthan, W. Liu, S. Tiwari, Multiobjective optimization test instances for the CEC 2009 special session and competition, Mech. Eng. (2008).
[13] L.M. Antonio, C.A.C. Coello, Use of cooperative coevolution for solving large-scale multiobjective optimization problems, in: Proceedings of the 2013 Congress on Evolutionary Computation, 2013, pp. 2758–2765.
[14] H. Zille, H. Ishibuchi, S. Mostaghim, Y. Nojima, A framework for large-scale multi-objective optimization based on problem transformation, IEEE Trans. Evol. Comput. 22 (2) (2018) 260–275.
[15] K. Deb, A. Saha, Multimodal optimization using a bi-objective evolutionary algorithm, Evol. Comput. 20 (1) (2012) 27–62.
[16] J.J. Liang, C.T. Yue, B.Y. Qu, Multimodal multi-objective optimization: a preliminary study, in: Proceedings of the 2016 Congress on Evolutionary Computation, 2016, pp. 2454–2461.
[17] K. Deb, S. Agrawal, A. Pratap, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, IEEE Trans. Evol. Comput. 6 (2) (2002) 182–197.
[18] Q. Zhang, H. Li, MOEA/d: a multiobjective evolutionary algorithm based on decomposition, IEEE Trans. Evol. Comput. 11 (6) (2007) 712–731.
[19] E. Zitzler, S. Künzli, Indicator-based selection in multi-objective search, in: Proceedings of Parallel Problem Solving from Nature, 2004, pp. 832–842.
[20] N. Khodadadi, S. Talatahari, A. Dadras Eslamlou, MOTEO: a novel multi-objective thermal exchange optimization algorithm for engineering problems, Soft Comput. 26 (2022) 6659–6684.
[21] M. Premkumar, P. Jangir, R. Sowmya, H.H. Alhelou, A.A. Heidari, H. Chen, MOSMA: multi-objective slime mould algorithm based on elitist non-dominated sorting, IEEE Access 9 (2021) 3229–3248.
[22] S. Mirjalili, P. Jangir, S. Saremi, Multi-objective ant Lion optimizer: a multi-objective optimization algorithm for solving engineering problems, Applied Intell. 46 (2017) 79–95.
[23] A. Divya, K. Kanak, C. Lenka, P. Pinank, M. Nikunj, J. Pradeep, Advancing truss structure optimization-a multi-objective weighted average algorithm with enhanced convergence and diversity, Results in Engineering 25 (2025) 104241.
[24] A. Vakili, H.M. Al-Khafaji, M. Darbandi, A. Heidari, N. Jafari Navimipour, M. Unal, A new service composition method in the cloud-based Internet of Things environment using a Grey wolf optimization algorithm and mapreduce framework, Concurr. Comput. Pract. Exp. 36 (16) (2024) e8091.
[25] K. Zanbouri, M. Darbandi, M. Nassr, A. Heidari, N. Navimipour, S. Yalcın, A GSO-based multi-objective technique for performance optimization of blockchain-based industrial internet of things, Int. J. Commun. Syst. 37 (15) (2024) e5886.
[26] S. Huband, P. Hingston, L. Barone, L. While, A review of multiobjective test problems and a scalable test problem toolkit, IEEE Trans. Evol. Comput. 10 (5) (2006) 477–506.
[27] S. Zapotecas-Martínez, A. García-Nájera, A. Menchaca-Méndez, Engineering applications of multi-objective evolutionary algorithms: a test suite of box-constrained real-world problems, Eng. Appl. Artif. Intell. 123 (2023) 106192.
[28] H. Tanabe, H. Ishibuchi, An easy-to-use real-world multiobjective optimization problem suite, Appl. Soft Comput. 89 (2020) 106078.
[29] D.H. Wolpert, W.G. Macready, No free lunch theorems for optimization, IEEE Trans. Evol. Comput. 1 (1) (1997) 67–82.
[30] Q. Zhu, Q. Lin, J. Chen, A gene-level hybrid search framework for multiobjective evolutionary optimization, Neural Comput. Appl. 30 (3) (2018) 759–773.
[31] H. Wang, Y. Fu, M. Huang, G. Huang, J. Wang, A hybrid evolutionary algorithm with adaptive multi-population strategy for multi-objective optimization problems, Soft Comput. 21 (20) (2016) 5975–5987.
[32] H. Bederina, M. Hifi, A hybrid multi-objective evolutionary optimization approach for the robust vehicle routing problem, Appl. Soft Comput. 71 (2018) 980–993.
[33] O. Dib, Novel hybrid evolutionary algorithm for bi-objective optimization problems, Sci. Rep. 13 (1) (2023).
[34] H. Zhang, J. Sun, K.C. Tan, Z. Xu, Learning adaptive differential evolution by natural evolution strategies, IEEE Trans. Emerg. Top. Comput. Intell. 7 (3) (2023) 872–886.
[35] H. Zhang, J. Sun, T. Bäck, Q. Zhang, Z. Xu, Controlling sequential hybrid evolutionary algorithm by q-learning, IEEE Comput. Intell. Mag. 18 (1) (2023) 84–103.
[36] F. Zou, G.G. Yen, L. Tang, C. Wang, A reinforcement learning approach for dynamic multi-objective optimization, Inf. Sci. 546 (2021) 815–834.
[37] M. Gao, X. Feng, H. Yu, X. Li, An efficient evolutionary algorithm based on deep reinforcement learning for large-scale sparse multiobjective optimization, Applied Intelligence 53 (18) (2023) 21116–21139.
[38] K. Li, T. Zhang, R. Wang, Deep reinforcement learning for multiobjective optimization, IEEE Trans. Cybern. 51 (6) (2021) 3103–3114.

[39] A. Heidari, N. Jafari Navimipour, M. Jabraeil Jamali, S. Akbarpour, Securing and optimizing IOT offloading with blockchain and deep reinforcement learning in multi-user environments, Wireless Netw. 31 (2025) 3255–3276.
[40] H. Li, Y. Tang, Y. Shui, J. Sun, A q-learning evolutionary multiobjective framework for multiobjective optimization with separable and interacting variables, in: Proceedings of the 2024 Congress on Evolutionary Computation, 2024, pp. 1–8.
[41] P. Cowling, G. Kendall, E. Soubeiga, A hyperheuristic approach to scheduling a sales summit, in: International Conference on the Practice and Theory of Automated Timetabling, 2000, pp. 176–190.
[42] E. Özcan, M. Misir, G. Ochoa, E.K. Burke, A reinforcement learning: great-deluge hyper-heuristic for examination timetabling, in: Modeling, Analysis, and Applications in Metaheuristic Computing: Advancements and Trends, 2012, pp. 34–55.
[43] H. Lu, X. Zhang, S. Yang, A learning-based iterative method for solving vehicle routing problems, in: International Conference on Learning Representations, 2019.
[44] J. Kallestad, R. Hasibi, A. Hemmati, K. Sörensen, A general deep reinforcement learning hyperheuristic framework for solving combinatorial optimization problems, Eur. J. Oper. Res. 309 (1) (2023) 446–468.
[45] K. Alakkari, B. Ali, T. Hameed, Optimizing hospital operational efficiency using AI: a multi-objective NSGA-II model for real-world medical data in Syria, Mesopotam. J. Artif. Intell. Healthc. 2025 (2025) 48–57.
[46] J. Katz, A. Plotkin, Applied Aerodynamics: a digital textbook (2010).
[47] H.L. Liu, F. Gu, Q. Zhang, Decomposition of a multiobjective optimization problem into a number of simple multiobjective subproblems, IEEE Trans. Evol. Comput. 18 (3) (2014) 450–455.
[48] R.S. Sutton, Reinforcement Learning: an Introduction, Cambridge: MIT Press, 2018.
[49] D. Silver, A. Huang, C.J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, et al., Mastering the game of GO with deep neural networks and tree search, Nature 529 (7587) (2016) 484–489.
[50] S. Levine, C. Finn, T. Darrell, P. Abbeel, End-to-end training of deep visuomotor policies, J. Mach. Learn. Res. 17 (39) (2016) 1–40.
[51] C.J. Watkins, P. Dayan, Technical note: Q-learning, Mach. Learn. 8 (1992) 279–292.
[52] V. Govindan, J. Jayaprakash, C. Park, J.R. Lee, I.N. Cangul, Optimization-based design and control of dynamic systems, Babyl. J. Math. 2023 (2023) 30–35.
[53] E.T. Khalid, A.J.Y. Aldarwish, A.A. Yassin, Challenges in automl and declarative studies using systematic literature review, Appl. Data Sci. Anal. 2023 (2023) 118–125.
[54] F.A. Hashim, Q.A. Zahraa, N.M. Hussien, Y.M. Mohialden, Enhanced priority-integrated mult winner voting software, Babyl. J. Artif. Intell. 2024 (2024) 27–33.
[55] Y. Tian, R. Cheng, X. Zhang, Y. Jin, Platemo: a MATLAB platform for evolutionary multi-objective optimization, IEEE Comput. Intell. Mag. 12 (4) (2017) 73–87.